

Technical Architecture & System Report

U1T02: Document Intelligence Asynchronous Pipeline

Author: Technical Architecture Team | **Date:** September 2026 | **Stack:** Docker Compose, FastAPI, Celery, Redis, PostgreSQL 16, Tesseract OCR

1. Executive Summary & Problem Context

Real-world document processing services cannot handle file ingestion synchronously. Documents arrive faster than they can be parsed, file sizes vary significantly, and heavy OCR operations cannot hold HTTP connections open without incurring network timeouts and socket exhaustion. This project implements a fully containerized, asynchronous **Document Intelligence Pipeline** using the **Claim-Check Pattern**, FastAPI, Celery, Redis, PostgreSQL 16, Tesseract OCR, and Celery Flower.

The entire stack is runnable via a single command: `docker compose up --build -d`.

2. Architecture & Claim-Check Pattern Design

The system decouples ingestion from execution by storing raw uploaded binary files directly in a shared storage volume (`/app/storage/uploads`) and enqueueing only a lightweight `job_id` claim-check ticket into Redis. This keeps message sizes in Redis extremely small (< 1 KB), eliminating key serialization penalties and memory bloat.

Step	Component	Action / Operation
1	Client -> FastAPI	POST <code>/api/v1/documents/upload</code> with document binary payload
2	FastAPI -> Storage	Writes binary to <code>/app/storage/uploads/{job_id}.ext</code> (Claim-Check)
3	FastAPI -> PostgreSQL	Creates job record with status <code>PENDING</code> and initial metadata
4	FastAPI -> Redis	Pushes lightweight <code>job_id</code> ticket to Redis task queue
5	Celery Worker	Pulls <code>job_id</code> from Redis, updates status to <code>PROCESSING</code>
6	Processing Engine	Executes format-specific parsing or Tesseract OCR
7	Worker -> PostgreSQL	Saves extracted text & metadata, updates status to <code>COMPLETED</code>
8	Client -> FastAPI	Polls GET <code>/api/v1/jobs/{job_id}/result</code> to retrieve extracted text

3. Technical Decision Justifications

- PostgreSQL 16 vs SQLite:** SQLite locks the entire database file during writes, causing lock contention under concurrent Celery workers. PostgreSQL 16 handles row-level locking, high concurrency, and native JSON metadata storage seamlessly.
- Celery + Redis Broker:** Redis provides sub-millisecond message broker speeds. Celery offers native task retry policies, visibility timeouts, late acknowledgments (`acks_late=True`), and Flower monitoring integration.
- PyMuPDF (fitz) vs Tesseract:** Vectorial PDFs are parsed via PyMuPDF in milliseconds ($\sim 0.08s$) with 100% precision. Tesseract OCR is triggered automatically as a fallback only when character density is < 15 chars/page (scanned documents).

4. Empirical Parser Benchmark (Part 3 Requirement)

Empirical benchmark evaluation performed on the sample document (U1T02.pdf - 2 Pages, 433 Words):

Parser Strategy	Time (sec)	Chars	Words	Accuracy	Recommendation
PyMuPDF (fitz)	0.087s	2,929	430	100%	Primary Choice for Native PDFs
pypdf	0.029s	2,927	430	99.8%	Fast Secondary Fallback
Tesseract OCR (spa+eng)	1.747s	2,900	430	98.5%	Essential for Scanned / Image PDFs

5. Resilience & Edge Case Handling

- **Unsupported Formats:** Rejected immediately at API boundary with HTTP 400 Bad Request.
- **Corrupted Files:** Caught in worker try/except block. Database record set to FAILED with full error_message.
- **Worker Abrupt Restart:** task_acks_late = True keeps tasks in Redis until successful completion, auto-redelivering to active workers if a worker dies mid-processing.